PROJECT DOCUMENTATION

HOTEL MANAGEMENT SYSTEM

Using Core PHP and MySQL

Student Name	Aakash Pandey
AF Code	AF05320000

Project Type: Web Application

Technology Stack: PHP, MySQL, HTML5, CSS3, Bootstrap, JavaScript

Development Environment: XAMPP / WAMP

Project Level: Beginner to Intermediate

Document Version: 1.0

1. Project Overview

1.1 Project Title

Hotel Management System

1.2 Introduction

The Hotel Management System is a web-based application developed using Core PHP and MySQL. It helps hotel staff manage room information, guest details, reservations, check-in/check-out records, and billing through a centralized system.

The application is intended to reduce manual paperwork, make guest and booking information easier to find, and help staff maintain accurate room availability and reservation records.

1.3 Problem Statement

Managing hotel rooms and reservations manually in registers or spreadsheets can lead to duplicate bookings, incorrect guest details, difficulty tracking room availability, and delays during check-in, check-out, and billing. The proposed system stores hotel information in a structured database and provides screens to create, view, update, search, and cancel records.

1.4 Objectives

• Maintain guest, room, reservation, and billing records in a centralized database.

• Implement Create, Read, Update, and Delete (CRUD) operations for appropriate records.

• Search guest records and reservations quickly.

• Track room types, room status, and booking dates.

• Validate user input and prevent overlapping reservations for the same room.

• Provide administrator login and access control.

• Reduce paperwork and improve record accuracy.

1.5 Project Scope

The initial version is designed for small hotels, guest houses, and lodging businesses. Core scope includes administrator login, room management, guest registration, reservation management, check-in/check-out status, and basic bill generation. Online payment gateways, customer self-service booking, and advanced analytics may be added later.

2. Functional Requirements

Module	Description
Admin Login	Allows authorized hotel administrators or staff to log in.
Dashboard	Shows summary counts such as rooms, guests, bookings, occupied rooms, and available rooms.
Room Management	Adds, views, updates, searches, and marks rooms as available, occupied, or under maintenance.
Guest Management	Stores guest name, contact information, email, and identification details where required.
Reservation / Booking	Creates bookings with check-in and check-out dates, assigned room, guest, and booking status.
Check-in / Check-out	Records guest arrival and departure and updates the room/booking status.
Billing	Calculates room charges based on room rate and stay duration; records bill amount and payment status.
Search	Searches rooms, guests, or reservations by room number, guest name, or booking ID.

Module	Description
Logout	Ends the administrator's session.

2.1 Non-Functional Requirements

• Security: Store password hashes and use prepared statements to reduce SQL-injection risk.

• Performance: Room and booking lists should load within a reasonable time.

• Usability: Use a simple, consistent, user-friendly interface.

• Maintainability: Organize PHP code into configuration, shared includes, and module folders.

• Reliability: Validate dates and required fields; handle database errors safely.

• Data integrity: Prevent conflicting reservations for the same room and dates.

3. Technology Stack

Technology	Purpose
HTML5	Creates the structure of forms and pages.
CSS3	Styles the user interface.
Bootstrap	Provides responsive layout and UI components.
JavaScript	Adds client-side interactions and basic validation.
Core PHP	Implements server-side processing and hotel business logic.
MySQL	Stores room, guest, booking, administrator, and bill records.
PDO / MySQLi	Connects PHP to MySQL; prepared statements should be used.
XAMPP / WAMP	Provides a local Apache and MySQL development environment.

4. System Architecture

The application follows a basic three-layer architecture.

• Presentation Layer: HTML, CSS, Bootstrap, and JavaScript display forms, tables, and dashboards.

• Application Layer: Core PHP validates requests, applies booking rules, handles login, and processes database operations.

• Database Layer: MySQL stores administrators, rooms, guests, reservations, and bills.

4.1 Request Flow

The administrator opens the application in a browser.

The browser sends a request to the PHP application.

PHP checks authentication and validates the submitted data.

PHP runs the required query against MySQL.

MySQL returns the result to PHP.

PHP displays a confirmation, record list, or controlled error message.

5. Database Design

5.1 Database Name

hotel_management

5.2 Main Tables

Table	Purpose / Important fields
admins	id (PK), username, password_hash — authorized staff login.
rooms	id (PK), room_number (unique), room_type, rate_per_night, status.
guests	id (PK), full_name, email, phone, id_reference (optional).
bookings	id (PK), guest_id (FK), room_id (FK), check_in, check_out, booking_status, created_at.
bills	id (PK), booking_id (FK), room_charges, extra_charges, total_amount, payment_status, created_at.

5.3 Relationships

• One guest can have multiple bookings over time (guests → bookings).

• One room can be associated with many bookings over time, but date-overlap rules must prevent simultaneous active bookings (rooms → bookings).

• This sample design assumes one bill per booking (bookings → bills).

5.4 Sample SQL Schema

```
CREATE DATABASE hotel_management;
USE hotel_management;

CREATE TABLE admins (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL
);

CREATE TABLE rooms (
  id INT AUTO_INCREMENT PRIMARY KEY,
  room_number VARCHAR(20) NOT NULL UNIQUE,
  room_type VARCHAR(50) NOT NULL,
  rate_per_night DECIMAL(10,2) NOT NULL,
  status ENUM('available','occupied','maintenance') NOT NULL DEFAULT 'available'
);

CREATE TABLE guests (
  id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150),
  phone VARCHAR(20) NOT NULL,
  id_reference VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  guest_id INT NOT NULL,
  room_id INT NOT NULL,
  check_in DATE NOT NULL,
  check_out DATE NOT NULL,
  booking_status ENUM('booked','checked_in','checked_out','cancelled') NOT NULL DEFAULT 'booked',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (guest_id) REFERENCES guests(id),
  FOREIGN KEY (room_id) REFERENCES rooms(id),
  CHECK (check_out > check_in)
);

CREATE TABLE bills (
  id INT AUTO_INCREMENT PRIMARY KEY,
  booking_id INT NOT NULL UNIQUE,
  room_charges DECIMAL(10,2) NOT NULL DEFAULT 0,
  extra_charges DECIMAL(10,2) NOT NULL DEFAULT 0,
  total_amount DECIMAL(10,2) NOT NULL DEFAULT 0,
  payment_status ENUM('unpaid','paid','partial') NOT NULL DEFAULT 'unpaid',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (booking_id) REFERENCES bookings(id)
```

```
);
```

5.5 Database Integrity Notes

Validate check-out date to be later than check-in date in PHP as well as in the database where supported. Before saving a booking, check whether the selected room has an overlapping booking whose status is not cancelled. The room status alone is not sufficient to prevent future-date booking conflicts.

6. Module Description

6.1 Admin Authentication

The administrator enters a username and password. The application verifies the stored password hash, starts a session, regenerates the session ID after successful login, and restricts management pages to authenticated users.

6.2 Room Management

Staff enter a room number, room type, nightly rate, and status. The system validates the values and saves the room. Staff can view and update room information; rooms in use should not be deleted if linked bookings need to be retained.

6.3 Guest Management

Staff register guest details such as full name and phone number, with email and identification reference as optional fields according to hotel policy. The guest record can be viewed, updated, and searched.

6.4 Reservation / Booking Management

Staff select a guest, room, check-in date, and check-out date. The system checks required fields, date order, and overlapping bookings before saving the reservation. A booking can be cancelled according to the hotel's process.

6.5 Check-in and Check-out

At check-in, staff confirm the reservation and update its status to checked-in. At check-out, the stay is finalized, the bill is calculated, and the room is made available after it has been cleaned and is ready for the next guest.

6.6 Billing

The system calculates room charges from the nightly rate and number of nights, adds any applicable extra charges, and stores the total and payment status. Actual payment processing is outside the initial scope unless a payment gateway is integrated.

6.7 Search

Staff can search for a room by room number, a guest by name or phone, and a reservation by booking ID or date. Search input must be validated and database queries should use prepared statements.

7. Suggested Project Folder Structure

```
hotel_management/
■■■ config/database.php
■■■ includes/header.php
■■■ includes/footer.php
■■■ includes/auth.php
■■■ auth/login.php
■■■ auth/logout.php
■■■ rooms/index.php
■■■ rooms/create.php
■■■ rooms/edit.php
■■■ rooms/delete.php
■■■ guests/index.php
■■■ guests/create.php
■■■ guests/edit.php
■■■ guests/search.php
■■■ bookings/index.php
■■■ bookings/create.php
■■■ bookings/edit.php
■■■ bookings/cancel.php
■■■ bookings/checkin_checkout.php
```

```
■■■ bills/view.php
■■■ bills/payment_status.php
■■■ assets/css/style.css
■■■ assets/js/script.js
■■■ dashboard.php
■■■ index.php
■■■ README.md
```

This is a suggested structure for a plain Core PHP project. Implemented filenames may differ depending on the final source code.

8. Development Methodology

Requirement Analysis: Identify hotel staff needs and define room, guest, booking, and billing features.

System Design: Plan the page layouts, booking workflow, and database relationships.

Database Development: Create the hotel_management database and tables.

Frontend Development: Build responsive forms, tables, and navigation.

Backend Development: Implement PHP authentication, validation, and database operations.

Validation and Security: Validate inputs, use prepared statements, and enforce access control.

Testing: Test room management, guest records, booking conflicts, and billing calculations.

Deployment: Configure the project on a suitable PHP web server.

9. Validation and Exception Handling

9.1 Input Validation

• Room number must be provided and must be unique.

• Room rate must be a valid non-negative amount.

• Guest name and phone number must be provided.

• Check-out date must be later than check-in date.

• A room must not have overlapping active bookings.

• Booking, guest, and room IDs must exist before update or cancellation.

• Only authorized staff can access restricted pages.

9.2 PDO Exception Handling Example

```
try {
    $pdo = new PDO(
        "mysql:host=localhost;dbname=hotel_management;charset=utf8mb4",
        $username,
        $password,
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
    );
} catch (PDOException $e) {
    error_log($e->getMessage());
    echo "Unable to connect to the hotel database.";
}
```

Database errors should be logged securely. Visitors should see a general error message, not raw database details. Use transactions when several database changes must succeed together.

10. Testing

Testing verifies that each hotel management feature works as expected.

10.1 Test Cases

Test ID	Test Scenario
TC01	Log in with valid credentials — dashboard opens.
TC02	Log in with invalid credentials — login error appears.
TC03	Add a room with valid details — room is saved.
TC04	Submit an empty required field — validation message appears.
TC05	Add a duplicate room number — duplicate is rejected.
TC06	Register a guest — guest record is saved.
TC07	Create a booking for an available room — booking is saved.
TC08	Try overlapping dates for the same room — conflicting booking is rejected.
TC09	Invalid check-out date — validation error appears.
TC10	Check in and check out a guest — booking status updates.
TC11	Generate a bill — charges and total are calculated.
TC12	Open protected page without login — access is denied or redirected.
TC13	Database connection fails — controlled error appears.

10.2 Types of Testing

• Unit Testing: Tests individual PHP functions, such as date validation and bill calculation.

• Integration Testing: Tests PHP-to-MySQL operations.

• Functional Testing: Verifies room, guest, booking, check-in/out, and billing workflows.

• Security Testing: Checks authentication, authorization, input handling, and SQL injection protection.

• User Acceptance Testing: Confirms the application meets hotel staff requirements.

11. Installation and Execution

11.1 Prerequisites

• XAMPP or WAMP installed.

• Apache and MySQL services.

• A modern web browser.

• A code editor such as Visual Studio Code.

11.2 Setup Steps

Install XAMPP and start Apache and MySQL.

Copy the project folder into the web root, for example C:\\xampp\\htdocs\\hotel_management.

Open phpMyAdmin.

Create the hotel_management database or run the SQL schema from Section 5.4.

Configure database host, database name, username, and password in config/database.php.

Create an administrator account using a securely hashed password.

Open http://localhost/hotel_management/ in the browser.

Test login, room management, guest registration, bookings, check-in/check-out, and billing.

Important: The documentation describes the intended design. Ensure the actual source code implements each listed module before claiming it is complete.

12. Limitations

• The initial version is intended for administrator or staff use only.

• Online customer booking is not included unless separately implemented.

• Payment gateway integration is not included.

• Housekeeping, restaurant services, inventory, and payroll are outside the initial scope.

• Advanced reports and multi-branch hotel management are not included.

13. Future Enhancements

• Customer-facing online booking portal.

• Online payment gateway integration.

• Email or SMS booking confirmations and reminders.

• Housekeeping and maintenance tracking.

• Room service and restaurant billing.

• Reports for occupancy, revenue, and bookings.

• Role-based access for receptionists, managers, and administrators.

• PDF/Excel invoice export.

• Mobile-friendly staff interface or API integration.

• Migration to Laravel for a more structured application.

14. Conclusion

The Hotel Management System demonstrates how Core PHP and MySQL can be used to manage essential hotel operations through a web-based application. Its proposed modules cover room records, guest information, reservations, check-in/check-out, and basic billing. The project applies key web-development concepts such as CRUD operations, database connectivity, validation, authentication, exception handling, and software testing. Additional features can be introduced as the hotel's needs grow.

15. References

• PHP Official Documentation — https://www.php.net/docs.php

• MySQL Official Documentation — https://dev.mysql.com/doc/

• Bootstrap Documentation — https://getbootstrap.com/docs/

• MDN Web Docs — https://developer.mozilla.org/

• OWASP Web Application Security Guidance — https://owasp.org/

Sample documentation for learning and reference. Update requirements, screenshots, implementation details, test results, and references to match the actual project before academic or professional submission.